FitPro

Online Fitness Training Platform

Java Servlets | JSP | JDBC | MySQL



Team Info

Aditya Singh
25scse1180171

Manik
25scse1180216

Harshit vats
25scse1180097

The problem and our solution



The problem

- Gym memberships are costly and time-bound
- Workouts are rarely tracked, so progress is invisible
- Trainers have no simple way to publish structured programs
- Members cannot see calories or trends in one place



Our solution

- One web platform for members and trainers
- Trainers publish Cardio, Strength and Yoga programs
- Members enrol, log sessions and see calories burned
- Dashboard turns raw logs into clear statistics

Key features by role



Member

- Register and log in securely
- Browse and filter workouts by type
- Enrol in programs
- Log minutes and body weight
- See calories, totals and a chart



Trainer

- Register and log in securely
- Publish programs with level and intensity
- Choose Cardio, Strength or Yoga
- Programs appear for every member
- Role-specific dashboard

Shared: password hashing, session login, input validation, friendly error messages

Technology stack



Frontend

JSP, JSTL, HTML5, CSS3, vanilla JavaScript



Backend

Java 11, Servlets 4.0, Filters, Listeners



Database

MySQL 8 accessed through JDBC



Build

Maven, packaged as a WAR file



Server

Apache Tomcat 9 (javax.servlet)

Chosen to cover the rubric directly: Core Java, JDBC database integration, and Servlets with web integration.

Layered MVC architecture



AuthFilter

Every /app/* request is checked for a logged-in session before it reaches a servlet.

Service layer

Validation and rules live here, so servlets stay thin and DAOs contain only SQL.

Listener and threads

AppContextListener stops the notification thread pool when the app is undeployed.

Project structure

```
fitpro/
|-- pom.xml
|-- README.md
|-- database/schema.sql
`-- src/main/
    |-- java/com/fitpro/
    |   |-- model/     entities (OOP)
    |   |-- dao/       JDBC classes
    |   |-- service/   business logic
    |   |-- servlet/   controllers
    |   |-- filter/ listener/
    |   `-- util/ exception/
    |-- resources/db.properties
    `-- webapp/ css, js, JSP views
```

29
Java classes

7
JSP pages

4
DB tables

6
Servlets

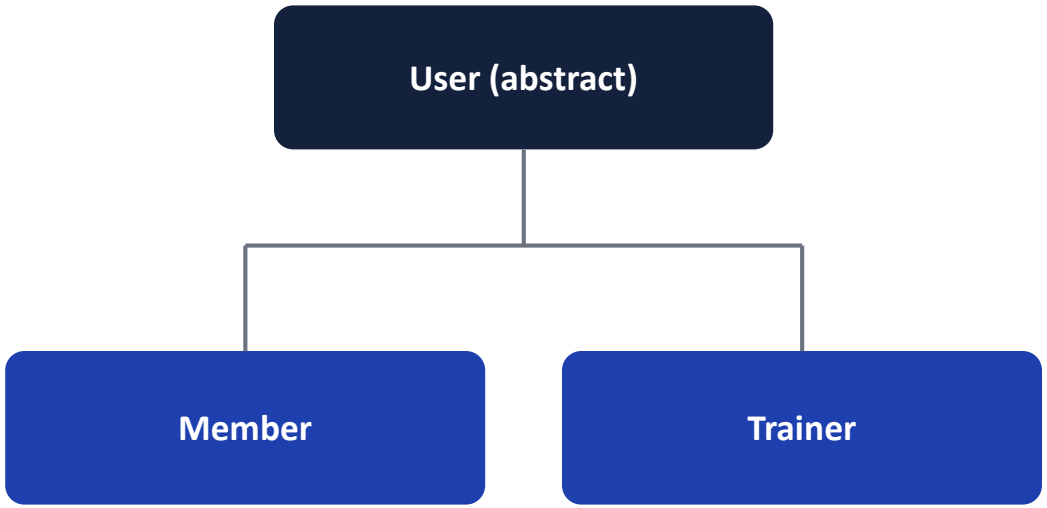
Database design

users	workouts	enrollments	progress_logs
id (PK)	id (PK)	id (PK)	id (PK)
full_name	title, description	user_id (FK)	user_id (FK)
email (UNIQUE)	type, level	workout_id (FK)	workout_id (FK)
password_hash	duration_min	enrolled_at	minutes, weight_kg
role: MEMBER / TRAINER	met_value	UNIQUE(user, workout)	calories
	trainer_id (FK)		logged_on

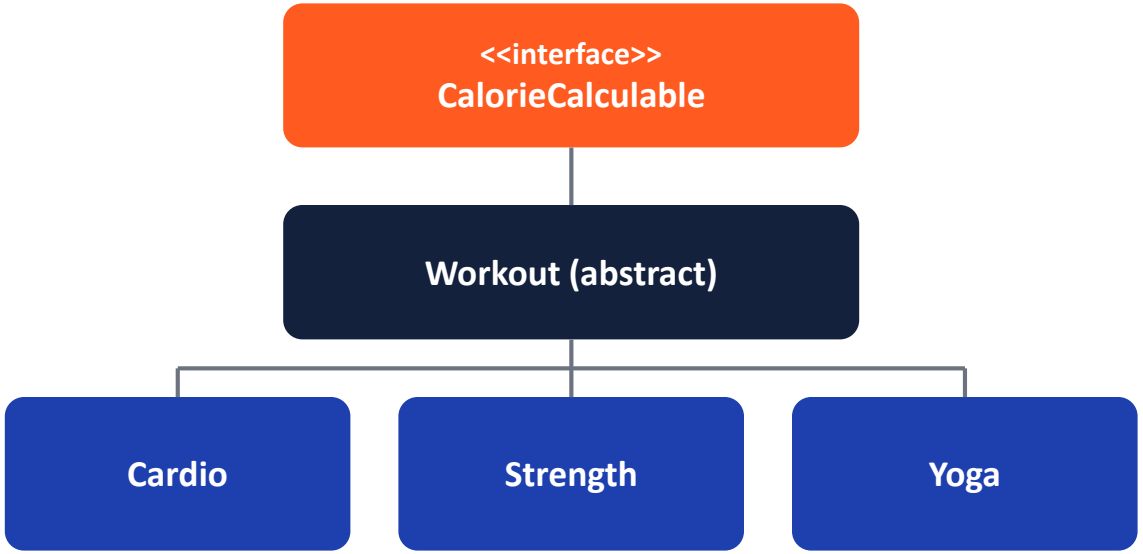
users 1 : N workouts (trainer publishes)
users N : M workouts through enrollments | users 1 : N progress_logs

OOP: inheritance, interfaces, polymorphism

Inheritance and abstraction



Interface and polymorphism



One call, different behaviour: workout.caloriesBurned(70 kg, 30 min)

MET 5 -> Cardio 183.8 kcal | Strength 202.1 kcal (+10% afterburn) | Yoga 165.4 kcal (-10%)

Collections, generics and exception handling

Collections and generics

- Generic interface BaseDAO<T, ID>
- List<Workout> for the catalogue
- Set<Integer> for fast enrolled checks
- TreeMap + streams: calories per workout
- Optional<T> instead of null returns

Exception handling

- Custom checked AppException
- DAOs wrap SQLException with clear messages
- try-with-resources closes every connection
- Servlets catch and show a friendly alert
- Login errors stay generic (no user hints)

```
public interface BaseDAO<T, ID> {
    Optional<T> findById(ID id) throws AppException;
    List<T> findAll() throws AppException;
}
```

Multithreading and synchronization

Why threads here?

- Registration returns instantly
- ExecutorService with 2 worker threads
- Shared counter guarded by synchronized
- Pool shut down cleanly by the listener
- Real gateway (email/SMS) can replace the stub




```
pool.submit(() -> {
    Thread.sleep(300);      // gateway delay
    int n = increment();    // thread-safe
    LOG.info("notification #" + n);
});

private synchronized int increment() {
    return ++sentCount;
}
```

JDBC and servlet integration



JDBC layer

- DBConnection loads the driver and settings once
- db.properties, overridable by environment variables
- PreparedStatement for every query
- try-with-resources for connections and results
- Four DAOs: User, Workout, Enrollment, Progress

URL	Servlet and purpose
/login	LoginServlet: authenticate
/register	RegisterServlet: sign up
/app/dashboard	DashboardServlet: statistics
/app/workouts	WorkoutServlet: list, enrol, publish
/app/progress	ProgressServlet: log sessions
/app/logout	LogoutServlet: end session

Every /app/* URL passes through AuthFilter first.

Security measures



Password hashing

PBKDF2-HMAC-SHA256 with a random salt. Plain passwords are never stored.



SQL injection

Every query uses PreparedStatement with bound parameters.



Cross-site scripting

All JSP output goes through c:out so user text is escaped.



Sessions and roles

Session regenerated at login, HttpOnly cookies, role checks on the server.

Rubric coverage

Criterion	Marks	Evidence in FitPro
Problem understanding and solution design	8	Role-based platform, layered MVC design
Core Java concepts	10	OOP, collections, generics, exceptions, threads
Database integration (JDBC)	8	DBConnection, 4 DAOs, PreparedStatement
Servlets and web integration	7	6 servlets, filter, listener, JSP and JSTL

33

marks addressed

Bonus: topics from the GUI rubric are also covered (collections, threads, DB classes).

Live demo flow and future scope

1. Register a Trainer and publish a program
2. Register a Member and browse workouts
3. Enrol in a program from the catalogue
4. Log a session and open the dashboard

Future scope

- CSRF tokens on all forms
- Admin approval for trainer accounts
- Video lessons and weekly goals
- Charts with Chart.js
- REST API / Spring Boot version

Thank you

https://github.com/adiithakurr110/FitPro-Fitness-Platform

